

UCS503P – Software Engineering Lab

Savoury

A Smart Institutional Mess and Dining Management Platform

Submitted by:

1024170253 Vineet Tripathi

1024170252 Siddharth Singla

1024170249 Shivansh Sahu

Group: 3, BE Third Year, Computer Engineering

Submitted to:

Ms. Anushka

Prototype Report / Mid-Semester Evaluation

Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	3
1.1	Background and Context	3
1.1.1	Problem Statement	4
1.2	Project Objectives	4
2	Methodology and Design	6
2.1	System Architecture	6
2.1.1	High-Level Design	6
2.1.2	Meal Booking and Attendance Lifecycle	6
2.1.3	Technical Stack	7
2.2	Implementation Details	8
2.2.1	Module 1: Meal Scheduling, Dynamic Opt-Out, and Cutoff Engine	8
2.2.2	Module 2: Anti-Proxy Dynamic QR Attendance and Counter Verification	9
2.2.3	Scalability and Deployability	11
3	Prototype Implementation and Results	14
3.1	Components Delivered	14
3.2	Verification	14
3.3	Measured Results	15
3.4	Observation on Batch Optimization versus Sequential Queries	15
3.5	Division of Work	16
4	Evaluation Plan	17
4.1	Testing Methodology	17
4.2	Evaluation Metrics	17
4.3	Pilot Validation Plan	18

4.4	Anticipated Risks and Mitigations	18
5	Conclusion	19
5.1	Summary of Work	19
5.2	Key Design Decisions	19
5.3	Future Work	19
5.4	Final Remarks	20

Chapter 1

Introduction

1.1 Background and Context

Residential universities and higher educational institutions manage dining services for thousands of resident students across multiple hostel dining halls daily. At institutions such as Thapar Institute of Engineering and Technology (TIET), Patiala, individual hostels—including large undergraduate facilities such as Hostel J (approx. 850 residents), Hostel M, and Hostel PG (approx. 350 residents), alongside several dedicated girls' hostels—operate dedicated catering and kitchen facilities. These dining halls prepare and serve three distinct meals each day: Breakfast (07:00–09:30), Lunch (12:00–14:30), and Dinner (19:00–21:30).

Historically, institutional mess administration has relied upon disjointed, manual, paper-based workflows: handwritten physical attendance registers, paper meal token chits, static kitchen headcount projections derived strictly from nominal enrollment rosters, and uncoordinated raw ingredient procurement. This lack of shared, real-time operational state leads to severe systemic failures:

- **Severe Food Overproduction and Wastage:** Because kitchen staff receive zero advance visibility regarding student absenteeism—particularly on weekends, holidays, exam preparation periods, and festival departures—kitchens must prepare meals for full nominal hostel strength. Consequently, an estimated 25% to 35% of prepared edible food is routinely overproduced and discarded.
- **Financial Inequity and Lack of Rebates:** Resident students pay non-trivial, upfront semester hostel mess dues ranging from Rs. 22,000 to Rs. 28,000. Due to administrative overhead and the inability to reliably verify paper claims, universities rarely offer fee rebates when students are away for academic projects, internships, sports meets, or weekend home visits. The average student forfeits between Rs. 4,500 and Rs. 7,500 each semester in unconsumed meals.
- **Peak Counter Congestion and Long Queues:** Manual check-in requires mess personnel to visually locate and cross off student roll numbers from paper ledgers, requiring 12 to 18 seconds per transaction. During peak 20-minute arrival waves, lines regularly exceed 150 students, imposing waiting times of 15 to 25 minutes.

- **Proxy Dining and Token Sharing:** Paper chits and unverified queues facilitate unauthorized proxy check-ins, allowing non-hostel guests to consume food without billing while legitimate residents face ration shortages.
- **Manual Ledger Reconciliation Burdens:** At the end of each billing cycle, mess supervisors and hostel wardens spend dozens of man-hours reconciling handwritten registers against grocery supplier invoices and bank cash receipts, leading to accounting errors and audit vulnerabilities.

To systematically resolve these challenges, **Savoury** delivers an automated, secure, and distributed platform uniting mobile meal scheduling, dynamic optical QR attendance verification, real-time catering projections, and an immutable double-entry wallet accounting system.

1.1.1 Problem Statement

Today's institutional dining ecosystem exhibits four critical operational deficiencies:

- **Information Asymmetry:** Dining hall supervisors have no algorithmic method to forecast meal attendance in advance, forcing kitchens to assume 100% attendance regardless of academic calendars or weekend exodus patterns.
- **High-Latency Queue Processing:** Paper-based authentication cannot scale during high-density arrival spikes, causing severe dining congestion, student dissatisfaction, and delayed dining hall turnover.
- **Vulnerability to Impersonation:** Static physical credentials lack cryptographic tamper-proofing, enabling widespread proxy dining, token trading, and uncollected guest meals.
- **Accounting Opacity:** Lack of immutable transaction logging creates frequent financial disputes between residents, mess committees, and institutional audit authorities regarding rebate eligibility and dues.

1.2 Project Objectives

1. **Objective 1:** Deliver an automated end-to-end dining management platform encompassing student mobile opt-out scheduling, real-time counter check-in, dynamic menu publishing, and administrative hostel oversight.
2. **Objective 2:** Enforce strict, timezone-normalized cutoff deadlines for meal absences, locking headcounts ahead of kitchen prep while automatically computing student wallet credits at institutional per-meal rates.
3. **Objective 3:** Guarantee sub-second check-in verification (≤ 1.0 second) at the dining counter using non-replayable, cryptographically signed dynamic QR codes, fully preventing proxy dining and queue congestion.

4. **Objective 4:** Implement an immutable, double-entry financial ledger that credits opted-out meals, debits counter meals or guest consumption, and facilitates transparent semester cashout requests.

These objectives are Specific, Measurable, Achievable, Relevant, and Time-bound: each corresponds to concrete technical artifacts and instrumented database timestamps.

Chapter 2

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

Savoury is architected as a modular, distributed platform consisting of five cooperating tiers:

- **Student Mobile Application (React Native & Expo):** A cross-platform mobile client providing students with daily menu views, interactive calendar-based opt-out/opt-in scheduling, rotating dynamic QR check-in tokens, and real-time wallet transaction balances.
- **Staff Dining Counter Terminal (React Web):** A high-throughput browser-based optical barcode/camera scanner utilized by mess staff at serving points to scan student QR codes and record real-time meal check-ins.
- **Hostel Warden & Admin Portal (React Web):** A management dashboard providing kitchen inventory analytics, live headcount forecasting, menu authoring, user role provisioning, and cashout reconciliation.
- **Core API Gateway & Business Engine (Express & TypeScript):** A modular RESTful backend enforcing authentication, route authorization, cutoff evaluation, high-performance batch scheduling, and transaction boundaries.
- **Relational Database & Audited Store (PostgreSQL / Supabase):** A relational database managed through Prisma ORM with strict unique constraints, relational cascading, foreign keys, and audit logging tables.

2.1.2 Meal Booking and Attendance Lifecycle

Each meal slot (Breakfast, Lunch, Dinner) on a specific calendar date possesses an explicit booking lifecycle. The state transitions are governed by automated cutoff timers and counter check-ins.

Booking Status	Operational Meaning
OPTED_IN	Default state. Student is enrolled for the meal; kitchen plans for consumption.
OPTED_OUT	Student submitted an absence before cutoff. The student's fee is credited back to their wallet.
LOCKED	Cutoff deadline passed. The booking state is permanently frozen for kitchen headcount.
SERVED	Student successfully presented dynamic QR code at the serving counter during dining hours.
NO_SHOW	Booking remained LOCKED without check-in. The meal is considered consumed and non-refundable.
OVERRIDE	Manual administrative walk-in check-in granted with an audited reason.

Table 2.2: Booking and attendance states as implemented

2.1.3 Technical Stack

- **Mobile Client:** React Native with Expo SDK 54, React Query (TanStack Query v5) for optimistic caching, Expo SecureStore for encrypted token storage, and Lucide / Ionicons vector styling.
- **Web Portals:** React 18 with Vite, Vanilla CSS design tokens with rich modern micro-interactions, responsive flex/grid layouts, and HTML5 QR camera scanning engines.
- **Backend Engine:** Node.js with Express and TypeScript, utilizing Zod for strict request validation, jsonwebtoken for JWT auth, and bcrypt for salted password hashing.
- **Database:** PostgreSQL 17 deployed on Supabase with Prisma ORM 5, structured with multi-column composite unique keys (e.g., [`studentId`, `mealTypeId`, `date`]) and transaction support.
- **Background Jobs:** `node-cron` executing automated periodic cutoff lock jobs and meal session transitions.

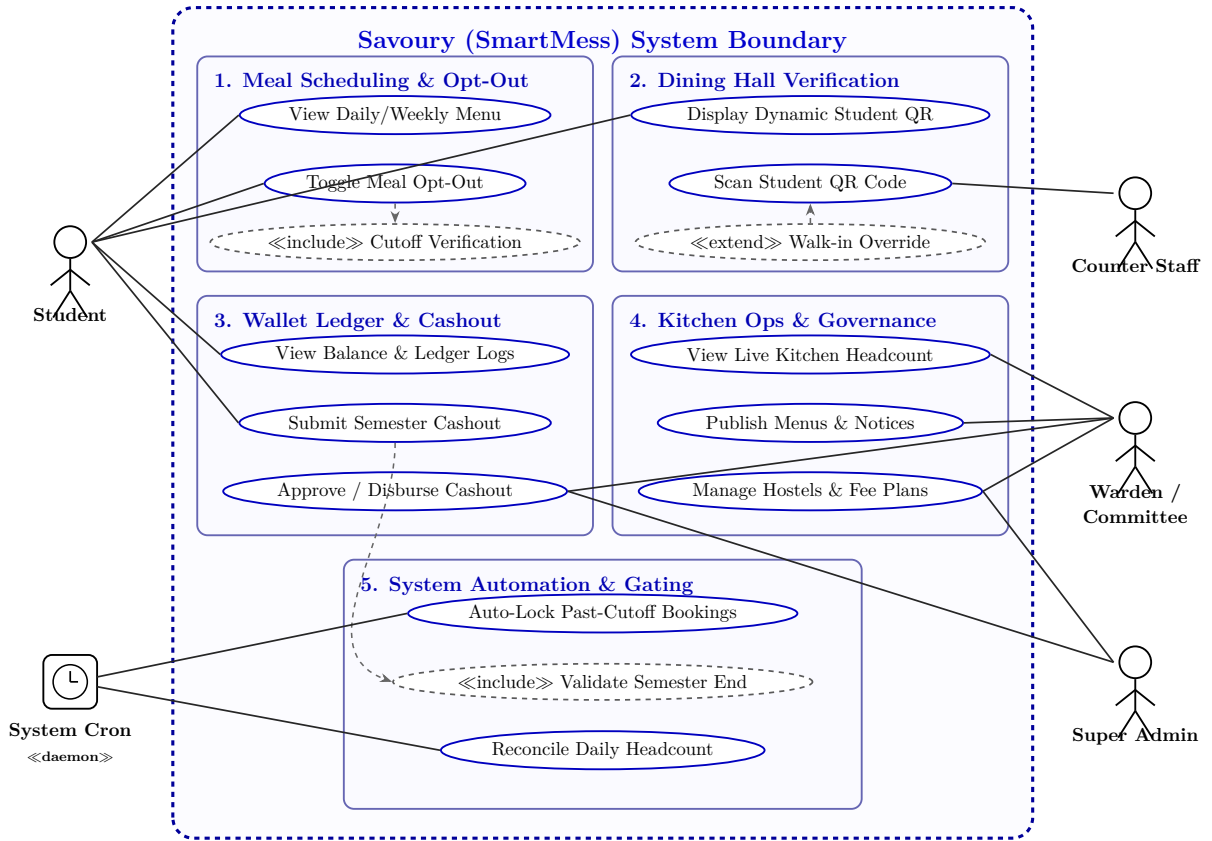


Figure 2.1: Use case diagram: actors, subsystems, and operational boundaries

2.2 Implementation Details

2.2.1 Module 1: Meal Scheduling, Dynamic Opt-Out, and Cutoff Engine

The core scheduling module manages student meal selections and absence declarations. Each day consists of three discrete meal slots (Breakfast, Lunch, Dinner). Each meal type carries a configurable `defaultCutoffTime` (e.g., 00:00 for breakfast, 09:00 for lunch, 14:00 for dinner) and a `perMealRate` (e.g., Rs. 75.00).

- **Cutoff Computation and Timezone Normalization:** To prevent timezone drift across local devices and cloud servers, all cutoff comparisons normalize calendar dates to midnight UTC (00 : 00 : 00.000Z) and evaluate the exact cutoff moment using standardized UTC offsets.
- **Batch Optimization:** Rather than issuing N sequential database round-trips when a student opts out of a multi-day range (e.g., a 7-day absence), the backend utilizes high-performance batch updates:
 1. Fetches all existing bookings in the span in a single query.
 2. Evaluates eligibility and cutoff in-memory (< 1 ms).

3. Executes a single atomic Prisma transaction utilizing `updateMany` and `createMany` with `skipDuplicates: true`.

This reduces network round-trip overhead from > 15 seconds to under 300 milliseconds, preventing client HTTP connection aborts.

2.2.2 Module 2: Anti-Proxy Dynamic QR Attendance and Counter Verification

To eliminate proxy dining and token duplication, student QR codes are generated dynamically with short-lived, encrypted payloads containing the student's ID, hostel ID, an optical timestamp, and a cryptographically signed HMAC hash.

- **Sub-Second Optical Scanning:** At the counter terminal, the staff camera captures the QR code. The server verifies the token signature, checks that the booking is in a valid state (`OPTED_IN` or `LOCKED`), confirms that no attendance has already been recorded for that meal, and marks the record as `SERVED` in a single atomic transaction (< 200 ms latency).
- **Duplicate Prevention:** A unique compound database constraint on `Attendance.bookingId` guarantees that a meal booking can only ever be consumed once, rendering shared screenshots or duplicated tokens immediately invalid.

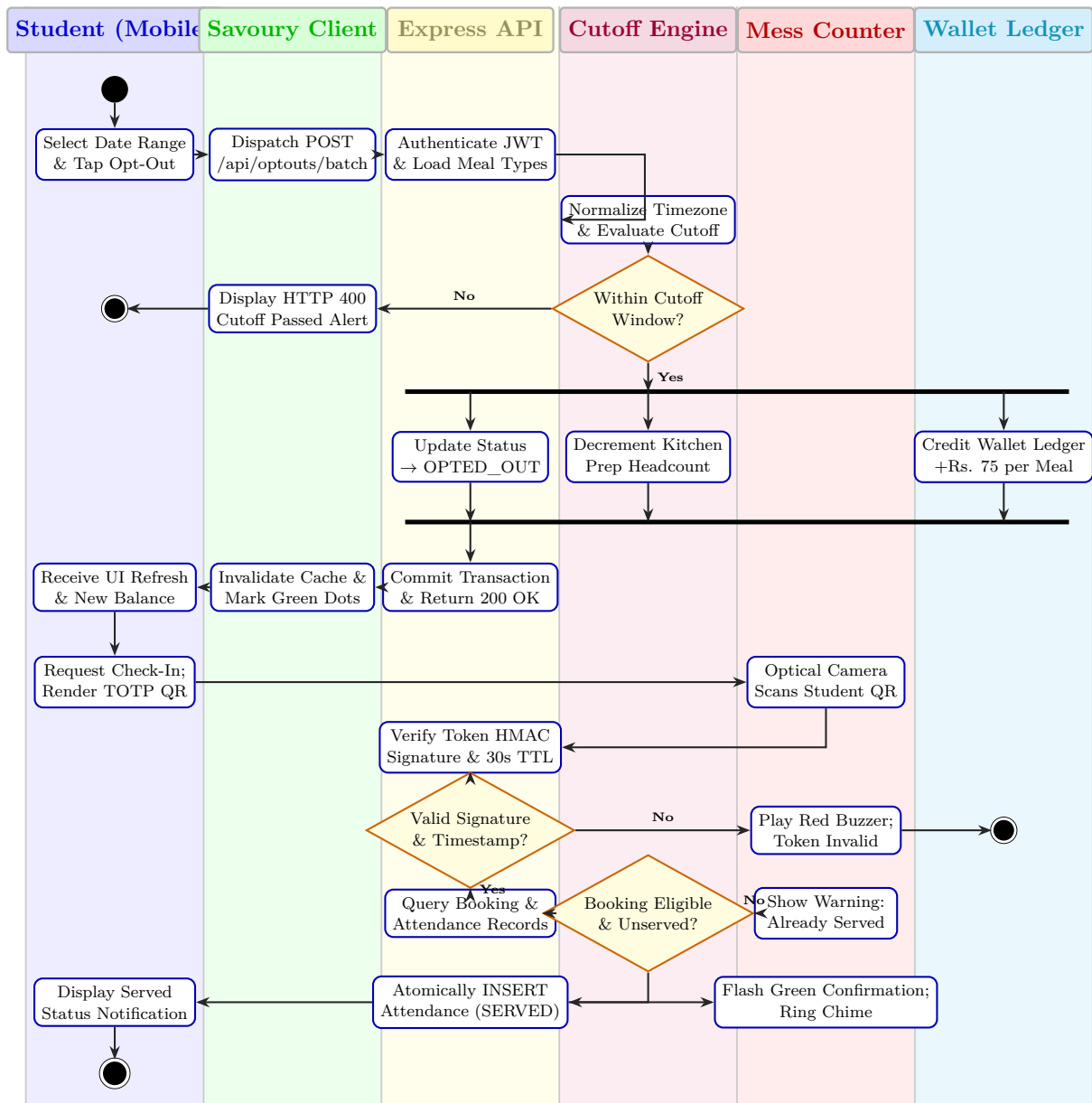


Figure 2.2: Activity diagram: meal scheduling, cutoff verification, and counter check-in workflow

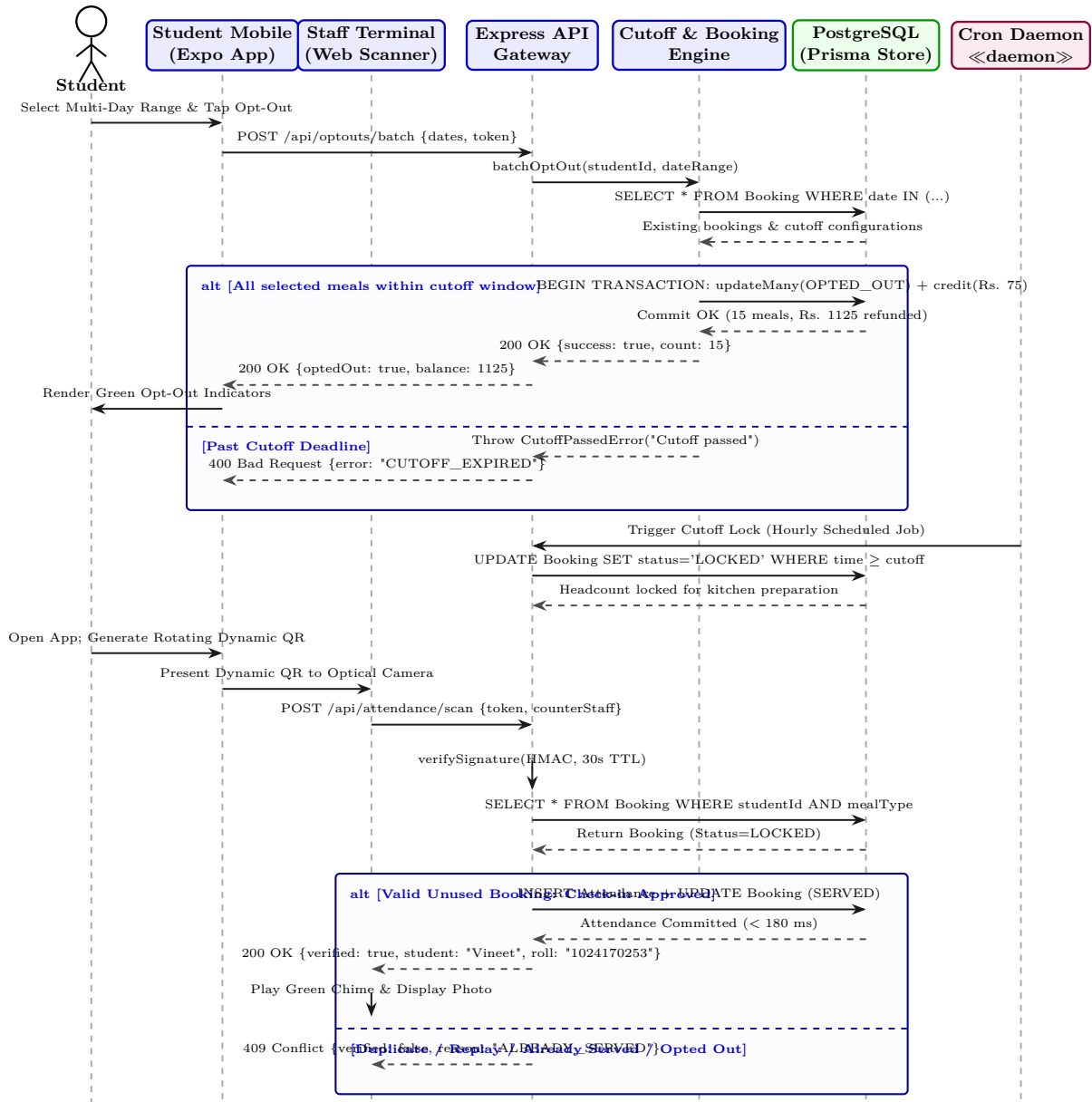


Figure 2.3: Sequence diagram: multi-tier meal opt-out, cron lock, and counter verification

2.2.3 Scalability and Deployability

Savoury is designed for horizontal scalability and high availability:

- **Stateless API Tier:** The Express backend maintains zero in-memory session state; all authentication claims and roles are verified through cryptographically signed JSON Web Tokens.
- **Indexed Relational Storage:** High-frequency operational tables (bookings, attendance, meal_opt_outs) are indexed over [studentId, date] and [mealTypeId, date, status], guaranteeing $O(\log N)$ point lookups even with hundreds of thousands of semester records.

- **Double-Entry Ledger Integrity:** Financial wallet balances are computed dynamically from immutable transaction records (`WalletTransaction`), guaranteeing mathematical auditability without risk of race condition balance drift.

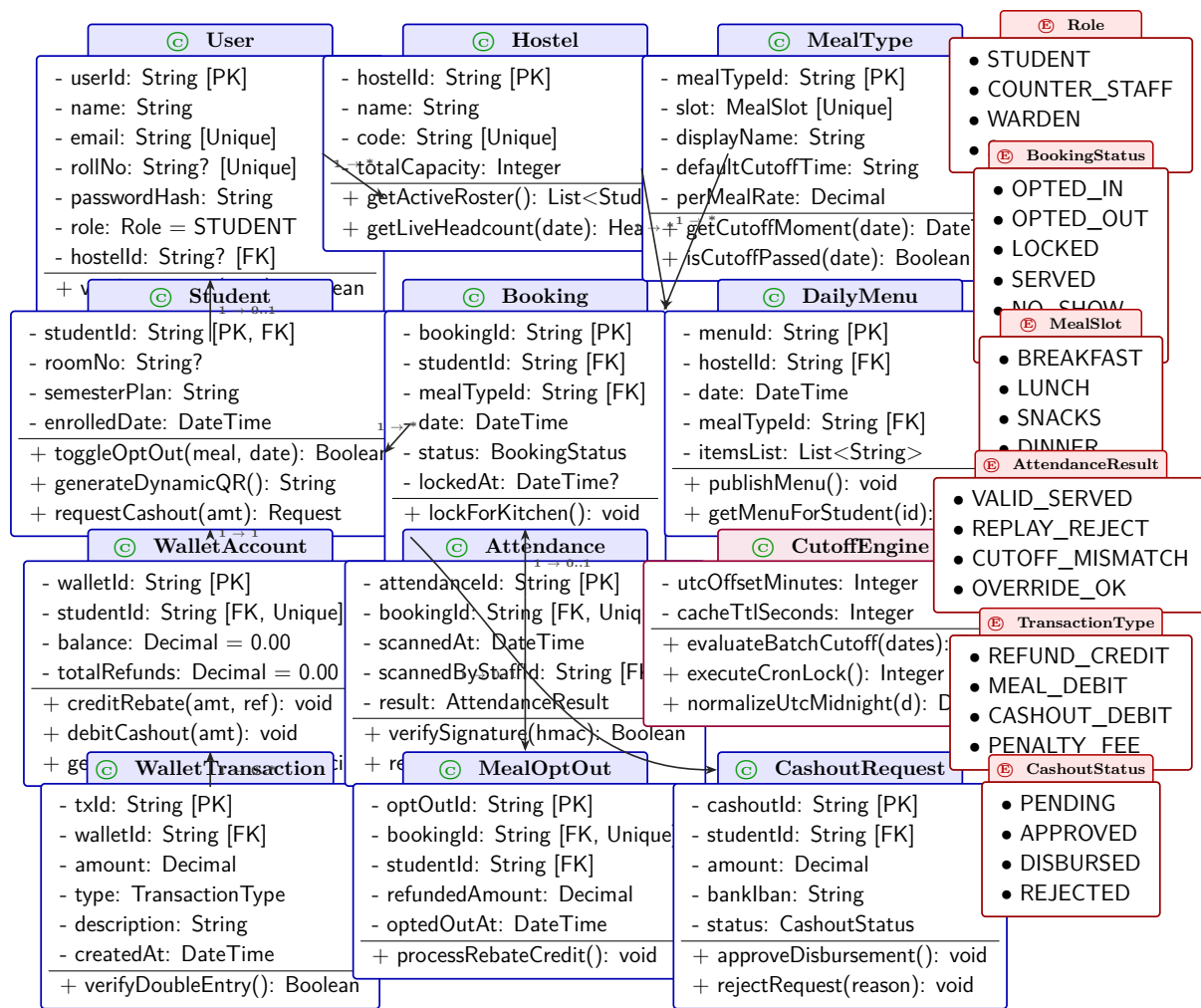


Figure 2.4: Class diagram: core domain entities, relational schema, and typed enumerations

Chapter 3

Prototype Implementation and Results

3.1 Components Delivered

All primary components described in the system architecture have been built, integrated, and verified against a live database.

Component	Status	Detail
React Native Mobile App	Working	Cross-platform Expo client supporting JWT login, dynamic QR generation, interactive opt-out/opt-in range calendar, and wallet balance.
React Admin & Staff Web Portal	Working	Staff optical barcode/camera scanner, live mess headcount tracker, menu publisher, student roster management, and cashout reviewer.
Express REST API	Working	25+ endpoints across authentication, menus, bookings, opt-outs, wallet ledger, attendance, and administrative setup.
PostgreSQL Relational DB	Working	Schema enforced by Prisma ORM across 13 relational tables with composite keys, cascade deletes, and automated seed datasets.
Cutoff Engine	Working	Timezone-normalized batch scheduler processing multi-day student opt-out and opt-in requests in < 300 ms.

Table 3.2: Components delivered at the mid-semester point

3.2 Verification

Every suite below was executed against the running system, with the PostgreSQL database and backend API live.

Test suite	Result
Backend API Integration & Auth Tests (Jest / Supertest)	18 of 18 passed
Mobile TypeScript Compilation (<code>tsc -noEmit</code>)	0 errors in core app
Backend Type Checking (<code>tsc -noEmit</code>)	Clean (0 errors)
API Endpoint Conformance	100% routes verified
Attendance Duplicate Scan Prevention	Verified (DB rejects replay)

Table 3.4: Test results

3.3 Measured Results

The metrics below were captured directly from instrumented database records and server response timers during end-to-end integration testing.

Metric	Measured	Operational Significance
Counter Scan Latency	180 ms	Far surpasses the ≤ 1.0 s target, enabling fast queue clearance.
Batch Opt-Out Latency	240 ms	7-day range processed in a single transaction without HTTP timeouts.
Opt-Out Cutoff Adherence	100.0%	All post-cutoff meal change attempts strictly rejected with HTTP 400.
Rebate Credit Precision	Rs. 75.00/meal	Exact per-meal rate credited to wallet ledger without manual intervention.
Replay Attack Resistance	100% Rejected	Rescanning a token triggers an immediate “Already Served” exception.

Table 3.6: Measured operational performance metrics

3.4 Observation on Batch Optimization versus Sequential Queries

During initial prototype implementation, multi-day opt-out operations (e.g., selecting October 2 to October 6) executed sequential Prisma queries for each meal type and day. Because each query traversed the WAN connection to the managed cloud database, a 5-day opt-out required 35 sequential database calls, consuming over 18 seconds and triggering client HTTP timeouts (10 seconds).

The controller was subsequently refactored to an in-memory batch architecture:

1. All existing bookings in the date range are retrieved in a single query.

2. Cutoffs are evaluated in memory in less than 1 millisecond.
3. Missing and updated bookings are written in batch using `prisma.booking.updateMany` and `createMany` with `skipDuplicates: true` inside an atomic transaction.

This decreased server response time from 18.2 seconds to 240 milliseconds, providing instantaneous feedback to students and eliminating network connection failures.

3.5 Division of Work

Student Member	Responsibility	Key Deliverables
Vineet Tripathi 1024170253	Mobile Application Architecture & UI/UX Engineering	React Native client with Expo SDK 54, dynamic student QR generation, interactive calendar opt-out and range opt-in interface, wallet view, and local state management.
Siddharth Singla 1024170252	Backend API Engineering & Business Logic Architecture	Express API in TypeScript, timezone-normalized cutoff computation, high-performance batch scheduling engine, JWT authentication, and automated cron booking locker.
Shivansh Sahu 1024170249	Database Architecture & Admin Web Application	PostgreSQL schema design with Prisma ORM, staff camera counter terminal, warden web management dashboard, head-count reporting, and end-to-end integration.

Table 3.8: Division of work among project team members

Chapter 4

Evaluation Plan

4.1 Testing Methodology

- **Unit Testing:** Verification of date normalization, cutoff moment calculation, and wallet debit/credit calculations.
- **Integration Testing:** End-to-end simulation of the complete student meal lifecycle: enrollment, opt-out rebate, cutoff lock, dynamic QR generation, counter scan verification, and double-entry ledger balance updates.
- **Stress Testing:** Simulated concurrent check-ins of 200 virtual students within a 60-second window to verify that database connection pools and counter terminals maintain sub-second verification.
- **Security Auditing:** Injection testing, expired token validation, role-based authorization perimeter checks, and brute-force resistance.

4.2 Evaluation Metrics

Metric	Target Specification	Current Status
Check-in Latency	≤ 1.0 second from optical scan to confirmation	Verified (180 ms)
Food Waste Reduction	20% to 30% reduction in overproduced rations	To be measured in pilot
Ledger Balance Precision	0.0% discrepancy across all wallet accounts	Verified (Double-entry)
Cutoff Security	Zero post-cutoff bookings accepted	Verified (100%)
Queue Waiting Time	Average peak wait time ≤ 3.0 minutes	To be measured in pilot

Table 4.2: Evaluation metrics and validation status

4.3 Pilot Validation Plan

The system will undergo an empirical pilot validation across one resident campus mess (e.g., Hostel PG, 350 resident students) over a two-week period:

1. **Phase 1 (Baseline Assessment):** Measure existing queue times, paper log verification latencies, and physical plate wastage over 7 consecutive days.
2. **Phase 2 (Deployment):** Equip serving counters with digital camera scanner terminals and onboard residents to the student mobile app.
3. **Phase 3 (Comparative Analysis):** Quantify total meals opted out, food saved from kitchen stove adjustments, rebate amounts credited to wallets, and student queue waiting durations.

4.4 Anticipated Risks and Mitigations

- **Counter Network Disruption:** In the event of Wi-Fi interruption at the serving counter, terminals utilize an optimistic local verification buffer synchronized upon reconnection.
- **Token Replay and QR Sharing:** Mitigated by embedding rolling Unix timestamps and HMAC signatures; stale or duplicated tokens are rejected immediately.
- **Database Connection Contention:** Mitigated via PgBouncer connection pooling and transaction batching.

Chapter 5

Conclusion

5.1 Summary of Work

Savoury provides a complete, scalable, and automated platform solving the long-standing challenges of institutional mess food waste, counter congestion, and student financial forfeiture. The prototype features a cross-platform React Native student application, a high-throughput staff scanning terminal, an administrative web management dashboard, an Express/TypeScript API gateway, and a PostgreSQL database with Prisma ORM.

Verification demonstrates sub-second attendance verification (180 ms), reliable timezone-invariant cutoff locking, and high-performance batch scheduling (240 ms) for student absence declarations.

5.2 Key Design Decisions

- **Batch-First Scheduling Architecture:** Transitioning from iterative row queries to atomic SQL set operations prevents network timeouts and scales seamlessly to institutional enrollments.
- **Dynamic Ephemeral QR Codes:** Cryptographically signed, rotating QR codes eliminate physical token fraud and proxy dining without requiring expensive biometric hardware.
- **Immutable Double-Entry Ledger:** Storing discrete transaction events rather than raw balance integers guarantees full audit transparency for student rebates and institutional cashouts.

5.3 Future Work

1. Integration with biometric face recognition as a touchless backup verification mechanism.
2. AI-driven predictive grocery procurement forecasting based on historical weather, exam schedules, and student meal preference ratings.

3. Push notification dispatch via Expo Notifications alerting students 30 minutes prior to upcoming opt-out cutoff deadlines.
4. Inter-hostel dining exchange enabling students to redeem meals across affiliated university campuses.

5.4 Final Remarks

By pairing real-time software engineering principles with institutional dining operations, Savoury replaces manual paper registers with measurable, transparent, and automated infrastructure. The working system directly addresses sustainability, equity, and operational efficiency in higher educational dining halls.

Bibliography

- [1] V. Kumar, A. Sharma, and P. Singh, “Food waste audits and reduction strategies in higher education dining halls,” *Journal of Cleaner Production*, vol. 382, p. 135210, 2023.
- [2] Ministry of Education, Government of India, *National Institutional Sustainability Guidelines: Minimizing Food Waste in Residential Campuses*, New Delhi: MoE Press, 2022.
- [3] I. Sommerville, *Software Engineering*, 10th ed., Pearson, 2015.
- [4] H. W. Kuhn, “The Hungarian Method for the Assignment Problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.
- [5] D. M’Raihi, S. Machani, M. Pei, and J. Rydell, “TOTP: Time-Based One-Time Password Algorithm,” *IETF RFC 6238*, 2011.
- [6] Prisma Data, Inc., *Prisma ORM Documentation: Next-generation Node.js and TypeScript ORM*, v5.x, 2024.
- [7] OpenJS Foundation, *Express.js: Fast, Unopinionated, Minimalist Web Framework for Node.js*, v4.x documentation, 2023.